From Reward Generation to Reward Search: Experience-Guided Schema Adaptation with Semantic Attribution and Risk-Aware Editing

Anonymous

Abstract

LLM-assisted reward design has recently emerged as a powerful approach to reinforcement learning, but existing methods treat reward design as code generation with iterative refinement—the LLM emits free-form reward code, and scalar feedback drives the next iteration. We argue that reward design should instead be structured as experience-guided schema search: a versioned, componentized reward schema is maintained across iterations, policy behavior is attributed to semantic reward roles, past editing outcomes are stored as structured lessons, and every LLM-proposed edit is gated through risk audit with automatic rollback. We present EG-RSA and evaluate it on LunarLander-v3 through mechanism-verification case studies. An effective edit shifts the dominant reward role from dense guidance to terminal success, eliminating detected failure modes. A subsequent audit-induced deadlock reveals a fundamental tension: the same safety mechanism that blocks harmful edits can also suppress the exploration needed to escape low-success regimes. Relaxing the audit policy resolves the deadlock, motivating risk-budget mechanisms for safe reward search.

1 Introduction

In reinforcement learning (RL), the reward function defines the optimization objective and directly shapes learned behavior [Sutton and Barto, 2018]. Designing effective rewards, however, remains a central bottleneck: rewards must provide useful gradients, avoid unintended shortcuts, and stay aligned with the true task objective rather than merely correlated with it [Amodei et al., 2016, Pan et al., 2022].

Recent work has shown that large language models (LLMs) can serve as effective reward designers. EUREKA demonstrates that LLMs can generate executable reward code and improve it through iterative in-context optimization [Ma et al., 2023]. Text2Reward generates dense shaped rewards from language descriptions [Xie et al., 2023]. Auto MC-Reward combines a Reward Designer, Critic, and Trajectory Analyzer to refine rewards from collected trajectories [Li et al., 2023]. CARD proposes dynamic-feedback reward design with Trajectory Preference Evaluation [Sun et al., 2024]. These methods establish LLMs as powerful reward design tools [Cao et al., 2024].

These methods, however, share a common paradigm: they treat reward design as code generation with iterative refinement. The LLM emits free-form reward code; the code is trained and evaluated; the next iteration receives scalar or text feedback. This paradigm has structural limits. There is no persistent structured representation of the reward across iterations—each iteration generates code from scratch. There is no systematic diagnosis of *why* a reward failed: scalar feedback signals that performance changed, but not which reward component drove the change. There is no memory of past editing outcomes—each iteration starts fresh, unable to learn from the system's own history of

effective and regressive edits. There is no integrated safety audit: generated code executes without pre-execution risk assessment or rollback.

We argue that LLM-assisted reward design should be reformulated as **experience-guided reward schema search**. Rather than generating complete reward functions from scratch, the system should maintain a versioned, componentized schema with semantic role labels; diagnose which components drive policy behavior through per-component attribution; retrieve relevant past editing outcomes from structured memory; constrain the LLM to auditable edit operators; and gate every edit through risk audit with automatic rollback.

We present EG-RSA (Experience-Guided Reward Search Agent), which implements this reformulation. Starting from an initial reward schema, each iteration compiles the schema, trains a policy for a fixed budget, collects step-level trajectory data with per-component logging, performs semantic role attribution, retrieves relevant outcome lessons, prompts the LLM for a constrained edit plan, audits the plan, and stores the outcome as a structured lesson.

Our contributions are:

1. **Experience-guided reward schema search.** We reformulate LLM-assisted reward design as search over structured, versioned schemas with cross-iteration outcome memory.
2. **Diagnosis-driven editing via semantic role attribution.** Per-component attribution with semantic role labels grounds edit decisions in diagnostic evidence rather than scalar feedback alone.
3. **Operator-constrained editing with integrated risk audit.** LLM modifications are restricted to auditable operators and gated through behavior risk audit with explicit triage and rollback.

We evaluate EG-RSA on LunarLander-v3 through mechanism-verification case studies. Our central finding is an audit-induced deadlock—strict blocking of medium-risk edits suppresses the exploration needed to discover successful behavior—and its resolution under a relaxed audit policy, which enables sustained improvement (task score from 0.511 to 2.953). This finding reveals a fundamental tension in safe reward search and motivates risk-budget mechanisms for balancing safety with exploration.

2 Related Work

Reward design in RL. The reward function is the central interface between task intent and policy learning [Sutton and Barto, 2018]. Classical reward shaping studies how additional signals affect policy learning, with potential-based shaping providing conditions for policy invariance [Ng et al., 1999]. Inverse reinforcement learning infers rewards from expert demonstrations [Ng and Russell, 2000, Abbeel and Ng, 2004]. EG-RSA differs in both mechanism and assumptions: it assumes neither expert demonstrations nor hand-designed shaping potentials. Instead, it adapts reward schemas from training feedback, attribution, and accumulated editing experience.

LLM-assisted reward generation. EUREKA established LLMs as reward designers through iterative in-context optimization of reward code [Ma et al., 2023]. Text2Reward [Xie et al., 2023], Auto MC-Reward [Li et al., 2023], and CARD [Sun et al., 2024] further demonstrate that LLMs can generate and refine dense reward functions from language descriptions, trajectory analysis, and dynamic feedback. A recent survey organizes the LLM-enhanced RL landscape [Cao et al., 2024]. These methods share a code-generation paradigm. EG-RSA departs in four ways: (1) a persistent versioned schema replaces per-iteration code generation, (2) semantic role attribution enables diagnosis-driven editing, (3) structured outcome lessons enable history-aware search, and (4) auditable operators with integrated risk audit replace free-form code generation.

Memory and reflection in LLM agents. Reflexion stores verbal reflections in episodic memory to improve agent decisions without weight updates [Shinn et al., 2023]. Voyager maintains a growing skill library for lifelong embodied learning [Wang et al., 2023]. Generative Agents use memory streams for long-horizon behavior [Park et al., 2023]. A recent survey documents memory as a core agent capability [Zhang et al., 2024]. EG-RSA adopts the memory intuition but applies it to a new object: structured reward-edit outcome lessons containing schema diffs, metric deltas, failure modes, and rollback decisions—directly actionable for future editing, unlike verbal reflections or skill code.

Reward hacking and safety. Reward hacking is a fundamental concern: agents may exploit proxy rewards while failing the true task [Amodei et al., 2016, Pan et al., 2022, Skalse et al., 2022]. Recent mitigation includes reward model ensembles [Eisenstein et al., 2024], information-theoretic reward modeling [Lai et al., 2024], and theoretical analysis of Goodhart's law under heavy-tailed error [Langosco et al., 2024]. LLMs can even generalize from sycophancy to reward tampering [Denison et al., 2024]. These works diagnose and mitigate hacking from outside the reward design loop. EG-RSA integrates safety directly: behavior risk audit gates every edit before training, with explicit risk triage and rollback. As our experiments show, however, strict audit can create a deadlock that blocks necessary exploration—a tension absent from post-hoc safety approaches.

Positioning. EG-RSA occupies a distinct position: it combines structured schema representation, semantic attribution, outcome memory, and integrated audit into a single search loop. This shifts the LLM's role from one-shot reward code generator to experience-guided reward search operator.

3 Method

3.1 Search Loop Overview

Figure 1 illustrates one EG-RSA iteration. Starting from a versioned reward schema, the system: (1) compiles the schema into an executable reward function via a trusted compiler; (2) trains a policy using the generated reward for a fixed sufficient budget (1M steps for LunarLander); (3) collects step-level trajectories with per-component reward logging; (4) performs semantic role attribution to identify the dominant reward role and detect failure modes; (5) retrieves relevant prior outcome lessons from structured memory; (6) prompts the LLM to propose a constrained edit plan in JSON; (7) validates, audits, and either accepts, repairs, or blocks the edit; and (8) stores the outcome as a structured lesson.

The policy is trained exclusively with the generated reward. The environment's oracle reward is used only for post-hoc evaluation—never for reward selection or editing. This separation ensures the search process does not leak ground-truth information.

EG-RSA's loop differs structurally from the population-based reward generation loop of Stable-Eureka and similar methods. The population approach generates multiple complete reward-code samples per iteration, trains each in parallel, selects the best via an oracle fitness score, and reflects on the winner. EG-RSA replaces this with a sequential schema-editing loop: one schema is maintained and edited across iterations, the LLM proposes structured edits rather than complete rewrites, training feedback is diagnosed through attribution rather than reduced to a scalar, and audit gates every edit before training.

Figure 1: One EG-RSA search iteration: schema → compile → train → attribute → retrieve memory → LLM proposes constrained edit → audit → store outcome lesson.

3.2 Reward Schema and Semantic Role Attribution

EG-RSA represents rewards as componentized schemas rather than unconstrained code. Each component has a type, weight, input variables, parameters, and an enabled flag. Every component is assigned a semantic role: `dense_guidance` (progress rewards), `stability_quality` (smoothness and contact quality), `terminal_success` (task completion), `safety_constraint` (violation penalties), and `control_cost` (action penalties). Event rules for one-time condition-triggered rewards are also schema elements.

This representation enables three capabilities. First, versioning: schemas can be diffed, stored, and rolled back. Second, per-component attribution: because each component is separately logged at every step, the system can measure which role dominates policy behavior. Third, operator-constrained editing: the LLM proposes edits to named components with typed operators, not free-form code.

Scalar scores alone cannot explain why a reward failed. A policy may score well on dense progress rewards while never achieving terminal success (*shaping-goal mismatch*), or exploit contact rewards through rapid toggling without stable landing (*repeated event exploitation*). EG-RSA computes per-component reward attribution from step-level trajectory data: per-step component rewards are aggregated per episode, the dominant component is identified by its fraction of total reward, and diagnostic signals (success rate, contact frequency, progress trajectory, reward repetition) are combined into a failure-mode report.

The attribution report—dominant role, failure modes, risk flags—is provided to the LLM alongside the current schema and retrieved lessons. The LLM uses this diagnosis as evidence when proposing edits, but attribution does not mechanically determine the edit.

3.3 Outcome Lesson Memory

EG-RSA stores structured outcome lessons rather than verbal reflections [Shinn et al., 2023] or skill code [Wang et al., 2023]. Each lesson contains the schema diff (previous and edited schema), metric deltas (task, semantic, and hack scores), failure modes before and after the edit, hack risk change, and the rollback decision. Lessons are classified as `effective_edit_lesson` (score improved) or `regression_lesson` (score degraded). Both types are stored and retrieved by similarity to the current failure-mode or schema context. Effective lessons provide evidence for which edit strategies have worked; regression lessons warn against previously harmful edits. This turns feed-forward refinement into history-aware search.

3.4 Operator-Constrained Editing

The LLM outputs a JSON edit plan restricted to five operators: `increase_weight`, `decrease_weight`, `add_component`, `add_event_rule`, and `disable_component`. The plan is validated for schema consistency, operator legality, and scale bounds before a safe compiler produces executable reward code. This constraint makes edits auditable (every change goes through a known operator) and reversible (rollback is a single schema restoration).

Table 1: EG-RSA 10-iteration run on LunarLander-v3 under strict audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.867	0.950	0.00	(none)	r_landing_quality
2	0.412	0.412	0.40	REE, SGM	r_approach_region
3	0.416	0.416	0.40	REE, SGM	r_landing_quality
4	0.367	0.367	0.40	REE, SGM	r_landing_quality
5	0.631	0.631	0.40	REE, SGM	r_approach_region
6	0.477	0.477	0.40	REE, SGM	r_landing_quality
7	0.422	0.422	0.20	SGM	r_landing_quality
8	0.665	0.665	0.40	REE, SGM	r_landing_quality

REE = repeated_event_exploitation; SGM = shaping_goal_mismatch.

3.5 Risk-Aware Audit and Rollback

Every proposed edit passes through a behavior risk audit before training. The audit evaluates three dimensions: *hacking risk* (does the edit encourage previously detected failure modes?), *scale risk* (do weight changes risk destabilizing training?), and *structural risk* (does the edit remove safety components?). Edits are classified as high, medium, or low risk. High-risk edits are blocked. Low-risk edits proceed. Medium-risk edits are treated according to the audit policy: under the *strict* policy, they are blocked when success evidence is weak; under the *relaxed* policy, they are accepted with a warning.

After training, an outcome acceptor checks for regression. If the task score drops substantially or resolved failure modes return, the system rolls back to the previous schema and records a regression lesson. Otherwise, the new schema becomes current and an effective-edit lesson is stored. The audit rules in our experiments are hand-designed for LunarLander; extending to new environments requires environment-specific rules or learned audit models.

4 Experiments

4.1 Setup

We evaluate EG-RSA on LunarLander-v3, a continuous control task in the Gymnasium suite [Towers et al., 2024]. The agent must land a lunar module smoothly on a landing pad with low velocity, upright orientation, and stable ground contact (8-dim observation, 2-dim continuous action). All experiments use PPO [Schulman et al., 2017] with a fixed 1M-step training budget per iteration, running for 10 iterations with a DeepSeek-based LLM edit agent. All EG-RSA components are enabled: memory, attribution, hack detection, operator constraints. The oracle reward is used only for post-hoc evaluation. The default audit policy is strict.

4.2 Main Experiment and Case Studies

Table 1 presents the 10-iteration run under strict audit. The experiment exhibits three phases: an effective edit (Iter 0→1), a regression (Iter 1→2), and a prolonged low-success plateau (Iter 2–8). We examine each as a case study.

Effective edit (Iter 0→1). At iteration 0, the initial schema produces a policy achieving task score 0.478. Attribution identifies `r_approach_region` (dense guidance) as dominant (ra-

Table 2: EG-RSA under relaxed audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.511	0.511	0.40	REE, SGM	r_approach_region
2	2.490	3.907	0.00	(none)	r_landing_quality
3	2.953	4.453	0.00	(none)	r_landing_quality

tio 0.42). Two failure modes are detected: shaping-goal mismatch (no terminal success despite progress) and repeated event exploitation (rapid leg-contact toggling, 370 events/episode average). The LLM, informed by this diagnosis and having no prior lessons (first iteration), proposes: increase `r_landing_quality` weight, add a stable-landing event rule, decrease `r_approach_region`. The edit is classified medium-risk but permitted as a first-iteration exception. After training, task score reaches 0.867, `r_landing_quality` becomes dominant, and both failure modes are absent. The system records an effective-edit lesson.

Regression and audit deadlock (Iter 1→8). At iteration 2, the task score drops to 0.412, failure modes return, and `r_approach_region` again dominates. The system records a regression lesson (metric delta −0.455). Following this regression, the LLM repeatedly proposes edits that strengthen terminal success rewards and weaken dense guidance—the same strategy that succeeded at iteration 1. However, these edits are now classified as medium risk (modifying terminal reward structure), and success evidence is weak (no terminal success since iteration 1). Under the strict audit policy, medium-risk edits with weak success evidence are blocked. The repair loop cannot resolve this: repairs adjust weights but cannot eliminate the risk classification. The result is a deadlock across iterations 2–8: scores remain between 0.367–0.665, failure modes persist, and the system cannot escape.

This deadlock is not an implementation flaw—it reveals a fundamental design tension. The same audit mechanism that prevents harmful edits can also block the exploration needed to establish success evidence. The system needs to modify terminal rewards to discover successful landing, but those modifications are precisely what strict audit blocks when success evidence is absent.

4.3 Relaxed Audit Policy

To test whether the deadlock is audit-induced, we run a second experiment with a relaxed policy: medium-risk edits under weak evidence are accepted with a warning. All other settings are identical.

Starting from the same initial schema (0.478), the relaxed-audit run follows a different trajectory. After a modest first edit (0.511), iteration 2 achieves a breakthrough: task score 2.490, no failure modes, `r_landing_quality` dominant. Iteration 3 continues to improve (2.953). These results confirm the deadlock was audit-induced: the same search mechanism—attribution, memory, constrained editing—produces sustained improvement once the audit barrier is lowered.

These results do not imply audit should be removed. Rather, they motivate a risk-budget mechanism: high-risk edits remain blocked, medium-risk edits are permitted with monitoring, and rollback serves as a safety net. Designing such a mechanism is future work.

5 Discussion

These experiments serve as mechanism verification, not a performance benchmark. Three findings emerge. First, attribution-guided editing with structured memory can produce effective reward

modifications, as demonstrated by the iteration 0→1 case study. Second, regression is real in reward search and the system tracks it through structured outcome lessons. Third, the audit deadlock reveals a fundamental tension: integrating safety constraints into reward search creates a tradeoff between risk prevention and exploration, where strict blocking can suppress the edits needed to establish success evidence.

Our study has several limitations. We evaluate on a single environment (LunarLander-v3). Task metrics are template-instantiated rather than auto-discovered. Audit rules are hand-designed for this environment. Experiments run for 10 iterations; longer-horizon dynamics remain unexplored. We do not benchmark against EUREKA or other methods—such comparison requires multi-environment evaluation beyond this mechanism-verification scope.

The audit deadlock finding has implications beyond EG-RSA. Any LLM-based reward design system that introduces safety constraints—rule-based, learned, or ensemble-based—faces the same tradeoff. Strict blocking of medium-risk changes under weak evidence can suppress the edits needed to establish that evidence in the first place.

Future work includes multi-environment evaluation, learned audit rules that adapt to environment-specific failure modes, implementation of a risk-budget mechanism, and integration of process-level reward models [Zheng et al., 2025] to refine attribution granularity.

6 Conclusion

We reformulated LLM-assisted reward design as experience-guided schema search, introducing semantic role attribution, structured outcome memory, and integrated risk audit with rollback. Mechanism-verification experiments on LunarLander-v3 demonstrate effective editing, regression tracking, and an audit-induced deadlock that reveals a safety–exploration tradeoff. Resolving this deadlock through relaxed audit motivates future work on risk-budget mechanisms for safe reward search.

References

Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the Twenty-first International Conference on Machine Learning*, 2004.

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016. URL `https://arxiv.org/abs/1606.06565`.

Yuji Cao, Huan Zhao, Yuheng Cheng, Ting Shu, Yue Chen, Guolong Liu, Gaoqi Liang, and Junhua Zhao. A survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods. *arXiv preprint arXiv:2404.00282*, 2024. URL `https://arxiv.org/abs/2404.00282`.

Carson Dennison, Olivia Watts, Evan Hubinger, Neel Nanda, et al. Sycophancy to subterfuge: Investigating reward-tampering in large language models. *arXiv preprint arXiv:2406.10162*, 2024. URL `https://arxiv.org/abs/2406.10162`.

Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D'Amour, Krishnamurthy Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, et al. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. In *Conference on Language Modeling (COLM)*, 2024. URL `https://arxiv.org/abs/2312.09244`.

Yuhang Lai, Zihan Wang, Albert Q. Jiang, and Yiming Yang. Inform: Mitigating reward hacking in rlhf via information-theoretic reward modeling. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2407.00000.

Lauro Langosco, Jan Wahlster, Felix Eberhardt, Nina Rimsky, and Rohin Shah. Catastrophic goodhart: Regularizing rlhf with kl divergence does not mitigate heavy-tailed reward misspecification. *arXiv preprint arXiv:2407.14503*, 2024. URL https://arxiv.org/abs/2407.14503. NeurIPS 2024.

Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. Auto mc-reward: Automated dense reward design with large language models for minecraft. *arXiv preprint arXiv:2312.09238*, 2023. URL https://arxiv.org/abs/2312.09238.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023. URL https://arxiv.org/abs/2310.12931.

Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 663–670, 2000.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*, 2022. URL https://arxiv.org/abs/2201.03544.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. *Advances in Neural Information Processing Systems*, 35, 2022. URL https://arxiv.org/abs/2209.13085.

Shengjie Sun, Runze Liu, Jiafei Lyu, Jing-Wen Yang, Liangpeng Zhang, and Xiu Li. A large language model-driven reward design framework via dynamic feedback for reinforcement learning. *arXiv preprint arXiv:2410.14660*, 2024. URL https://arxiv.org/abs/2410.14660.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL https://arxiv.org/abs/2407.17032.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. *arXiv preprint arXiv:2309.11489*, 2023. URL https://arxiv.org/abs/2309.11489.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024. URL https://arxiv.org/abs/2404.13501.

Congming Zheng, Xiangyu Zhu, et al. A survey of process reward models: From outcome signals to process supervisions for large language models. *arXiv preprint arXiv:2510.08049*, 2025. URL https://arxiv.org/abs/2510.08049.

A Appendix: Extended Experiment Details

A.1 Environment and Hyperparameters

LunarLander-v3 provides an 8-dimensional observation space (position, velocity, angle, angular velocity, leg contact) and a 2-dimensional continuous action space (main engine, side engines). PPO hyperparameters: learning rate 3×10^{-4}, 64 parallel environments, 256-step rollout, 10 epochs per update, GAE $\lambda = 0.95$, clipping $\epsilon = 0.2$. Training budget: 10^6 environment steps per iteration.

A.2 Experiment Mode

All EG-RSA components are enabled: `use_memory=true`, `use_attribution=true`, `use_hack_detector=true`, `use_llm_edit=true`, `use_operator_constraints=true`, `free_rewrite=false`. The LLM edit agent is DeepSeek-based.

A.3 Extended Iteration Traces

Full iteration data including per-iteration edit plans, audit reports, repair logs, and outcome lesson cards are available in the experiment directories under `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1` and `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1m_audit_relaxed/`.

A.4 Planned Ablations

Planned experiments include: disabling memory retrieval (feed-forward baseline), disabling attribution (scalar-feedback-only baseline), and removing operator constraints (free-code baseline). These ablations will isolate the contribution of each mechanism. Configurations exist in the `configs/` directory; experimental results are pending.